

Session Event Fan-Out

Session Event Fan-Out

Overview

This document defines the **Session Event Fan-Out Layer** for your event-driven, Redis-backed, polyglot agent system. It ensures that **all agent-emitted events** (logs, partial output, final output, state changes, approvals) are forwarded to the correct **per-session SSE channel**:

```
session:{sessionId}:events
```

This is essential for:

- Frontend SSE streaming
- Workflow orchestration
- Real-time UI updates
- Multi-agent pipelines
- Human-in-the-loop workflows

Without fan-out, the frontend receives no events.

Folder Structure

```
/gateway
/events
  fanout.ts
/redis
  sub.ts
/routes
  sessions.ts
```

File: gateway/events/fanout.ts

```
import { redisPub } from "../redis/pub";
import { RedisEventType } from "../../redis-schema/eventTypes";

interface AgentEvent {
  type: RedisEventType;
  sessionId: string;
  taskId?: string;
  timestamp: number;
  payload: any;
}

export async function fanOutEvent(event: AgentEvent) {
  if (!event.sessionId) return;

  const channel = `session:${event.sessionId};events`;

  await redisPub.publish(channel, JSON.stringify(event));
}
```

File: gateway/redis/sub.ts

```
import Redis from "ioredis";
import { fanOutEvent } from "../events/fanout";
import { normalizeEvent } from "../events/normalizeEvent";

export const redisSub = new Redis({ host: "localhost", port: 6379 });

export function initRedisSubscriber(app: any) {
  redisSub.subscribe("agent.logs");
  redisSub.subscribe("agent.events");

  redisSub.on("message", async (channel, message) => {
    const raw = JSON.parse(message);
    const event = normalizeEvent(channel, raw);

    // Log for debugging
    app.log.info({ event }, "Received agent event");

    // Fan-out to session channel
    await fanOutEvent(event);
  });
}
```

File: gateway/routes/sessions.ts

```
import { FastifyInstance } from "fastify";
import { createEventStream } from "../sse/eventStream";

export default async function sessionsRouter(app: FastifyInstance) {
  app.get("/:sessionId/events", async (req, res) => {
    const { sessionId } = req.params as { sessionId: string };
    createEventStream(sessionId, res);
  });
}
```

Integration Notes

Gateway Responsibilities

- Subscribe to agent.logs and agent.events

- Normalize events
- Fan-out events to session:{sessionId}:events
- Expose SSE endpoint for frontend

Agent Harness Responsibilities

- Emit events with correct sessionId
- Publish logs, partial output, final output, approvals

Frontend Responsibilities

- Connect to /sessions/:sessionId/events
- Route events into RxJS bus
- Update UI in real time

Workflow Engine Responsibilities

- Subscribe to session events
- Trigger node execution and orchestration

Summary

This Session Event Fan-Out Layer provides:

- Deterministic routing of agent events
- Real-time SSE streaming
- Full integration with Gateway, Agent Harness, and Workflow Engine
- Support for multi-agent pipelines and human-in-the-loop workflows

It completes the event propagation layer required for your MVP.